NYU Tandon School of Engineering

Department of Computer Science and Engineering

Snickr: Database Design and Web Application Report

CS6083 — Final Project

Elliot Tuchscherer

May 7, 2026

https://github.com/et6239/DBFinalProject

Contents

1 Introduction

Snickr is a web-based team collaboration system modeled after Slack. Users sign up with an email address, choose a username and nickname, and join workspaces. Within each workspace, members communicate through channels that are either public (open to all workspace members), private (invite-only), or direct (exactly two users). Every action in the system is timestamped, and access control is enforced at the application layer rather than through per-user database accounts.

This report covers both parts of the CS6083 final project:

- **Part 1** (Sections 2–6): the entity-relationship model, relational schema, SQL queries, and sample data that define the database design.
- **Part 2** (Sections 7–12): the full browser-accessible web application built on top of the Part 1 schema using Django 6.0.3 and PostgreSQL. This section covers the application architecture, session management, access control strategy, AJAX features, security measures, and a user guide.

2 Entity-Relationship Design

The ER diagram in Figure 1 uses Chen notation. Rectangles represent entities, ellipses represent attributes (double-border ellipses are primary keys), dashed ellipses are relationship attributes, and diamonds are relationships.

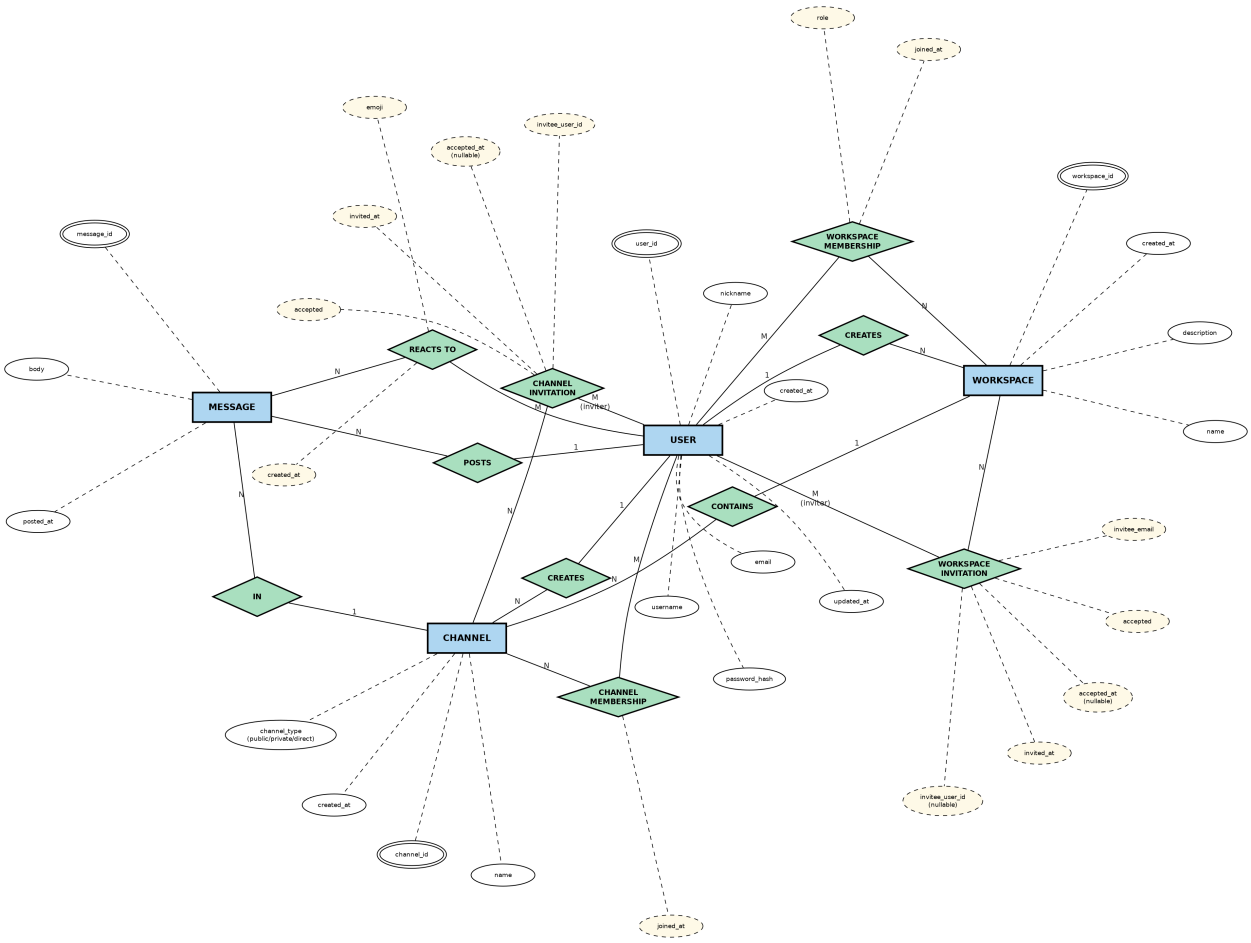

Figure 1: Chen-notation ER diagram for Snickr (updated for Part 2)

The four core entities are **USER**, **WORKSPACE**, **CHANNEL**, and **MESSAGE**. The key relationships are:

- A user *creates* a workspace (1:N) and a channel (1:N).
- Users participate in workspaces through a *WORKSPACE MEMBERSHIP* relationship (M:N) carrying a role (`admin` or `member`) and a join timestamp.
- Users can be invited to workspaces via *WORKSPACE INVITATION* (M:N), which stores the inviter, the invitee's email and optional user ID, the invite timestamp, and an accepted flag with timestamp.
- A workspace *contains* channels (1:N).
- Channel access is tracked through *CHANNEL MEMBERSHIP* (M:N) and *CHANNEL INVITATION* (M:N), which follow the same pattern as the workspace equivalents.
- A user *posts* messages (1:N), and each message belongs to exactly one channel via *IN* (N:1).
- Users *react to* messages (M:N) through the **MESSAGE REACTION** relationship, which was added in Part 2. Each reaction carries an `emoji` attribute and a `created_at` timestamp. A unique constraint on `(message_id, user_id, emoji)` ensures each user can apply each emoji to a given message at most once.

3 Relational Schema

Each ER entity and M:N relationship translates to a separate table. Two custom ENUM types are defined first: `member_role` (`admin`, `member`) and `channel_kind` (`public`, `private`, `direct`).

users

Column	Type	Constraints	References
user_id	SERIAL	PRIMARY KEY	
email	VARCHAR(255)	NOT NULL, UNIQUE	
username	VARCHAR(100)	NOT NULL, UNIQUE	
nickname	VARCHAR(100)	NOT NULL	
password_hash	VARCHAR(255)	NOT NULL	
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

workspace

Column	Type	Constraints	References
workspace_id	SERIAL	PRIMARY KEY	
name	VARCHAR(100)	NOT NULL	
description	TEXT		
created_by	INT	NOT NULL	users(user_id)
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

workspace_membership

Composite primary key on (`workspace_id`, `user_id`).

Column	Type	Constraints	References
workspace_id	INT	NOT NULL	workspace(workspace_id)
user_id	INT	NOT NULL	users(user_id)
role	member_role	NOT NULL, DEFAULT 'member'	
joined_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

workspace_invitation

Column	Type	Constraints	References
invitation_id	SERIAL	PRIMARY KEY	
workspace_id	INT	NOT NULL	workspace(workspace_id)
invited_by	INT	NOT NULL	users(user_id)
invitee_user_id	INT	nullable	users(user_id)
invitee_email	VARCHAR(255)	NOT NULL	
invited_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	
accepted	BOOLEAN	NOT NULL, DEFAULT FALSE	
accepted_at	TIMESTAMP	nullable	

channel

Unique constraint on (`name`, `workspace_id`).

Column	Type	Constraints	References
channel_id	SERIAL	PRIMARY KEY	
name	VARCHAR(100)	NOT NULL	
workspace_id	INT	NOT NULL	workspace(workspace_id)
channel_type	channel_kind	NOT NULL	
created_by	INT	NOT NULL	users(user_id)
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

channel_membership

Composite primary key on (`channel_id`, `user_id`).

Column	Type	Constraints	References
channel_id	INT	NOT NULL	channel(channel_id)
user_id	INT	NOT NULL	users(user_id)
joined_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

channel_invitation

Column	Type	Constraints	References
invitation_id	SERIAL	PRIMARY KEY	
channel_id	INT	NOT NULL	channel(channel_id)
invited_by	INT	NOT NULL	users(user_id)
invitee_user_id	INT	NOT NULL	users(user_id)
invited_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	
accepted	BOOLEAN	NOT NULL, DEFAULT FALSE	
accepted_at	TIMESTAMP	nullable	

message

Column	Type	Constraints	References
message_id	SERIAL	PRIMARY KEY	
body	TEXT	NOT NULL	
channel_id	INT	NOT NULL	channel(channel_id)
posted_by	INT	NOT NULL	users(user_id)
posted_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	

message_reaction

Added in Part 2 to support emoji reactions. A unique constraint on `(message_id, user_id, emoji)` ensures idempotency — a user cannot apply the same emoji twice to the same message.

Column	Type	Constraints	References
id	SERIAL	PRIMARY KEY	
message_id	INT	NOT NULL	message(message_id)
user_id	INT	NOT NULL	users(user_id)
emoji	VARCHAR(10)	NOT NULL	
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	
UNIQUE (message_id, user_id, emoji)			

4 Keys and Constraints

Primary keys. Every table has a primary key. The core entity tables (`users`, `workspace`, `channel`, `message`, `workspace_invitation`, `channel_invitation`, `message_reaction`) use a surrogate `SERIAL` key. The two membership tables use composite primary keys: `(workspace_id,`

`user_id)` and `(channel_id, user_id)`, which naturally enforce that a user can hold at most one membership record per workspace or channel.

Foreign keys. All references to `users`, `workspace`, `channel`, and `message` are enforced as foreign key constraints. The one exception is `workspace_invitation.invitee_user_id`, which is nullable to allow invitations to be sent to email addresses that do not yet correspond to a registered user.

Other constraints. `UNIQUE` is applied to `users.email`, `users.username`, `(channel.name, channel.workspace_id)`, and `(message_reaction.message_id, message_reaction.user message_reaction.emoji)`. `NOT NULL` is applied to all columns that must always be present. `DEFAULT NOW()` is used for every timestamp column so that the application does not need to supply timestamps explicitly. The `member_role` and `channel_kind` ENUM types enforce that only valid values are stored in those columns.

5 SQL Queries and Results

Placeholder values are noted in comments. All queries were tested against the sample data described in Section 6.

C1 — Create a New User Account

```
INSERT INTO users (email, username, nickname, password_hash)
VALUES ('grace@nyu.edu', 'grace', 'GraceG', '$2b$12$hashed_grace');
```

```
INSERT 0 1
```

C2 — Create a Public Channel (with Authorization Check)

The `INSERT ...SELECT ...WHERE EXISTS` pattern serves as the authorization check. If the user is not a workspace member the inner `SELECT` returns no rows and neither `INSERT` executes. The CTE auto-enrolls the creator as the channel's first member.

```
-- user_id=2 (bob) creates 'devops' in workspace_id=1 (Engineering)
WITH new_channel AS (
    INSERT INTO channel (name, workspace_id, channel_type, created_by)
    SELECT 'devops', 1, 'public', 2
    WHERE EXISTS (
        SELECT 1 FROM workspace_membership
        WHERE workspace_id = 1 AND user_id = 2
    )
    RETURNING channel_id, created_by
)
INSERT INTO channel_membership (channel_id, user_id)
SELECT channel_id, created_by FROM new_channel;
```

```
INSERT 0 1
```

C3 — Administrators for Each Workspace

```
SELECT w.name AS workspace, u.username, u.email
FROM workspace w
JOIN workspace_membership wm ON w.workspace_id = wm.workspace_id
JOIN users u                  ON wm.user_id     = u.user_id
WHERE wm.role = 'admin'
ORDER BY w.name, u.username;
```

```
 workspace   | username |     email
-------------+----------+---------------
 Engineering | alice    | alice@nyu.edu
 Marketing   | bob      | bob@nyu.edu
(2 rows)
```

C4 — Stale Pending Public Channel Invites

Counts users invited to a public channel more than 5 days ago who have not yet joined. Run against workspace_id = 1 (Engineering).

```
SELECT c.name AS channel, COUNT(ci.invitation_id) AS
   pending_invites_over_5_days
FROM channel c
LEFT JOIN channel_invitation ci
       ON c.channel_id  = ci.channel_id
      AND ci.accepted   = FALSE
      AND ci.invited_at < NOW() - INTERVAL '5 days'
WHERE c.workspace_id = 1 AND c.channel_type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;
```

```
 channel | pending_invites_over_5_days
---------+-----------------------------
 backend |                           1
 devops  |                           0
 general |                           2
(3 rows)
```

C5 — Messages in a Channel (Chronological)

```
-- channel_id = 1 (#general, Engineering)
SELECT m.message_id, u.username AS author, m.body, m.posted_at
FROM message m
JOIN users u ON m.posted_by = u.user_id
```

```
WHERE m.channel_id = 1
ORDER BY m.posted_at ASC;
```

```
 message_id | author |                      body                       |
    posted_at
------------+--------+-------------------------------------------------+-------------------

          1 | alice  | Welcome everyone to the Engineering workspace |
             2026-02-01 10:06:00
          2 | bob    | Thanks Alice, happy to be here.               |
             2026-02-05 09:10:00
          3 | alice  | Reminder: stand-up is at 10am.                |
             2026-02-10 08:00:00
          4 | bob    | The new API design is perpendicular to the... |
             2026-02-15 11:00:00
          5 | alice  | Good point Bob. Lets sync this afternoon.     |
             2026-02-15 11:05:00
          6 | dave   | Sprint planning moved to Thursday.            |
             2026-03-01 09:00:00
(6 rows)
```

C6 — All Messages Posted by a User

```
-- user_id = 1 (alice)
SELECT w.name AS workspace, c.name AS channel, m.body, m.posted_at
FROM message m
JOIN channel   c ON m.channel_id   = c.channel_id
JOIN workspace w ON c.workspace_id = w.workspace_id
WHERE m.posted_by = 1
ORDER BY m.posted_at ASC;
```

```
  workspace  |   channel    |              body                |
     posted_at
-------------+--------------+----------------------------------+--------------------

 Engineering | general      | Welcome everyone to the...       | 2026-02-01
    10:06:00
 Engineering | general      | Reminder: stand-up is at 10am.   | 2026-02-10
    08:00:00
 Engineering | general      | Good point Bob. Lets sync this.. | 2026-02-15
    11:05:00
 Engineering | backend      | Left some comments - the auth... | 2026-02-18
    10:30:00
 Engineering | secret-proj  | Kicking off the stealth project  | 2026-02-20
    09:05:00
 Engineering | dm-alice-bob | Hey Bob, can you take a look...  | 2026-03-05
    16:00:00
(6 rows)
```

C7 — Accessible Messages Containing a Keyword

Returns messages containing `perpendicular` that the user can access, meaning they hold membership in both the message's workspace and its specific channel.

```
-- user_id = 1 (alice), keyword = 'perpendicular'
SELECT w.name AS workspace, c.name AS channel,
       u.username AS author, m.body, m.posted_at
FROM message m
JOIN channel   c ON m.channel_id   = c.channel_id
JOIN workspace w ON c.workspace_id = w.workspace_id
JOIN users     u ON m.posted_by    = u.user_id
WHERE m.body ILIKE '%perpendicular%'
  AND EXISTS (
      SELECT 1 FROM workspace_membership wm
      WHERE wm.workspace_id = c.workspace_id AND wm.user_id = 1
  )
  AND EXISTS (
      SELECT 1 FROM channel_membership cm
      WHERE cm.channel_id = m.channel_id AND cm.user_id = 1
  )
ORDER BY m.posted_at ASC;
```

```
  workspace  |   channel    | author |           body            |
     posted_at
-------------+--------------+--------+---------------------------+---------------------

 Engineering | general      | bob    | The new API design is...  | 2026-02-15
     11:00:00
 Engineering | backend      | alice  | Left some comments - the  | 2026-02-18
     10:30:00
 Engineering | secret-proj | carol  | The architecture here...  | 2026-02-21
     10:10:00
(3 rows)
```

The fourth `perpendicular` message (in Marketing's `#general`) is correctly excluded because alice holds no membership in the Marketing workspace.

6 Test Data

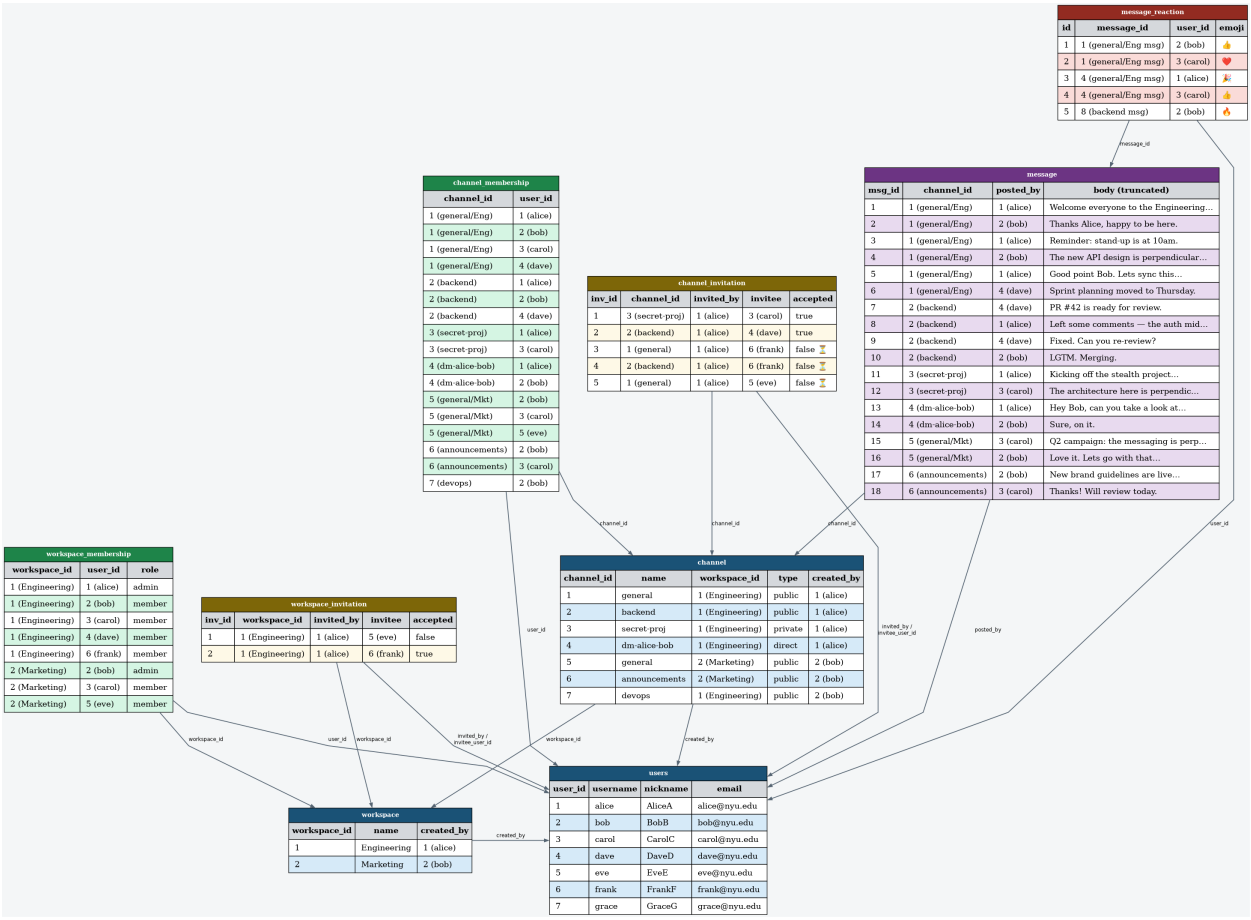

Figure 2: Data instance diagram showing all sample records and FK relationships (updated for Part 2)

The sample dataset was constructed to exercise all seven queries with non-trivial results.

Users. Six users are seeded with `PBKDF2` password hashes (password `"password"` for all) so that any account can be used to log in during development and testing. A seventh user (grace) is inserted by query C1.

Workspaces and channels. The Engineering workspace contains four channels covering all three channel types: two public (`#general`, `#backend`), one private (`#secret-proj`), and one direct (`dm-alice-bob`). Marketing contains two public channels. A seventh channel (`#devops`) was created by query C2.

C4 test cases. Three channel invitations were left in a pending state with `invited_at` values 6 to 10 days in the past: frank was invited to `#general` and `#backend`, and eve was invited to `#general`. These produce counts of 2 and 1 respectively.

C7 test cases. Four messages contain the word "perpendicular," spread across Engineering's `#general`, `#backend`, `#secret-proj`, and Marketing's `#general`. When the query runs as alice, only the first three are returned because alice has no access to the Marketing workspace.

Reactions. The `message_reaction` table is populated at runtime. The data diagram shows a representative set of five sample reactions to illustrate the structure.

7 Part 1 Assumptions

1. **Single application account.** The web application connects to the database under one shared account. Access restrictions are enforced at the application layer via cookies, not through database permissions.
2. **Auto-enrollment on creation.** When a user creates a workspace or channel, a membership row is inserted immediately alongside the new entity row, so the creator is always a member of what they create.
3. **Direct channels.** A direct channel always has exactly two members. This is enforced at the application layer rather than through a database constraint.
4. **Public channel joins.** Any workspace member may join a public channel directly. `channel_invitation` rows are used only for private channels.
5. **Unregistered invitees.** `workspace_invitation.invitee_user_id` is nullable so that a workspace admin can invite someone by email before that person has created an account. Once they register, the application links their `user_id` back to the invitation row.
6. **No deletion or editing.** Messages and channels cannot be deleted or edited. The schema has no `deleted_at` or `edited_at` columns.
7. **Passwords.** Only the hashed password is stored. In Part 2, Django's `PBKDF2-SHA256` algorithm is used via `make_password` / `check_password`.
8. **Timestamps in UTC.** All timestamps use the database server clock via `DEFAULT NOW()` and are stored in UTC.
9. **Channel name uniqueness.** Channel names are unique within a workspace but not globally, enforced by the composite unique constraint on `(name, workspace_id)`.
10. **Role promotion.** Promoting a member to admin is modeled as an update to the `role` column in `workspace_membership`. No separate administrator table is needed.

8 Part 2: Web Application Architecture

8.1 Technology Stack

Component	Choice
Web framework	Django 6.0.3 (Python 3, isolated project `.venv`)
Database	PostgreSQL (`snickr` database, TCP connection on `localhost:5432`)
DB driver	psycopg v3 (`psycopg[binary]`)
Frontend	Server-rendered HTML with vanilla JavaScript for AJAX
Templating	Django templates (`templates/` directory)
Static files	Django `staticfiles` (no bundler)
Session store	Django database-backed sessions (`django_session` table)

Django 6 was chosen specifically because it introduced native support for composite primary keys via `CompositePrimaryKey`, allowing the ORM to map directly onto the Part 1 schema's composite-PK membership tables without adding surrogate `SERIAL` columns.

8.2 Project Layout

The application follows the standard single-app Django layout:

```
snickr/           Django project settings and root URL conf
core/             Single Django app
  models.py       ORM models (all managed=False, mapping to Part 1 schema)
  views.py        All view functions
  urls.py         URL patterns
  auth.py         login_required decorator and get_current_user helper
templates/
  base.html       Shared layout, navigation, global CSS
  core/           Per-view templates
finalProjSchema/ Part 1 artefacts (schema.sql, queries.sql, diagrams, report
   )
```

8.3 URL Structure

Table 1 lists all URL routes and the view that handles each.

Table 1: URL routing table

Pattern	View
`/`	`dashboard`
`/register/`	`register`
`/login/`	`login`

Pattern	View
/logout/	logout
/profile/	profile
/workspaces/new/	workspace_new
/workspaces/<id>/	workspace
/workspaces/<id>/invite/	workspace_invite
/workspaces/<id>/members/<uid>/remove/	member_remove
/workspaces/<id>/members/<uid>/promote/	member_promote
/invitations/	invitations
/invitations/<id>/accept/	invitation_accept
/invitations/<id>/decline/	invitation_decline
/workspaces/<id>/channels/	channel_list
/workspaces/<id>/channels/new/	channel_new
/workspaces/<id>/members/search/	workspace_members_search
/channels/<id>/	channel
/channels/<id>/join/	channel_join
/channels/<id>/invite/	channel_invite
/channel-invitations/<id>/accept/	channel_invitation_accept
/channel-invitations/<id>/decline/	channel_invitation_decline
/search/	search
/channels/<id>/messages/json/	channel_messages_json (AJAX)
/channels/<id>/post/	message_post (AJAX)
/messages/<id>/react/	message_react (AJAX)

9 Session Management

9.1 Design Choice: Manual Authentication

Django ships with a built-in `django.contrib.auth` module that provides a `User` model, login/logout helpers, and a `@login_required` decorator. Snickr deliberately avoids it. The decision reflects two constraints of this project:

1. The schema was designed in Part 1 independently of Django. The `users` table structure differs from Django's `auth_user` table (different column names, no `is_staff`/`is_superuser` flags, etc.).
2. Adopting `django.contrib.auth` would require either migrating to its schema or maintaining a parallel user model, both of which would obscure the Part 1 relational design.

Instead, authentication is implemented manually using Django's generic session framework (`django.contrib.sessions`).

9.2 Session Lifecycle

Login. After verifying credentials, the `login` view stores two values in the server-side session:

```
request.session['user_id'] = user.user_id
request.session['username'] = user.username
```

Django serialises the session to the `django_session` database table and sends a `sessionid` cookie to the browser. All subsequent requests carry this cookie; the session middleware looks up the server-side record to re-hydrate the session dictionary.

Logout. `request.session.flush()` deletes the server-side session row and clears the cookie in one call, preventing session fixation.

Registration. On successful account creation, the user is immediately logged in (session is populated) so they are not forced to log in twice.

Invite linkage. When a new user registers, the application checks whether any pending workspace invitations were sent to their email address before they had an account. If so, it updates `invitee_user_id` on those rows so the user can see and accept them immediately after logging in.

9.3 The `login_required` Decorator

A custom decorator in `core/auth.py` guards every view that requires an authenticated user:

```
def login_required(view_fn):
    @wraps(view_fn)
    def wrapper(request, *args, **kwargs):
        if not request.session.get('user_id'):
            return redirect('login')
        return view_fn(request, *args, **kwargs)
    return wrapper
```

If `user_id` is absent from the session the request is redirected to `/login/` before the view body executes. A companion helper, `get_current_user(request)`, performs a single database lookup by `user_id` and is called at the top of each protected view.

10 Access Control Strategy

10.1 Layered Enforcement

Access control is enforced in multiple layers so that no single omission exposes data. The checks are applied in order, from coarsest to finest:

1. **Session gate.** `@login_required` rejects any unauthenticated request before any database query is made.
2. **Workspace membership gate.** Every workspace-scoped and channel-scoped view verifies that the current user holds a row in `workspace_membership` for the relevant workspace. Users who are not workspace members receive a 403 response even for public channels within that workspace.
3. **Channel membership gate.** Private and direct channels additionally require a row in `channel_membership`. Users who lack channel membership receive a 403 even if they are workspace members.
4. **Role gate.** Admin-only operations (invite to workspace, remove member, promote member) check that `workspace_membership.role = 'admin'` for the current user. The private-channel invite flow additionally restricts the action to the channel creator.
5. **HTTP method gate.** State-mutating endpoints (join channel, post message, react to message) are decorated with Django's `@require_POST`, returning 405 for any other HTTP method. This prevents CSRF-style abuse via browser prefetch or link crawling.

10.2 Channel Type Enforcement

The three channel types create distinct access policies:

Type	Who can read	How to join
public	Any workspace member	Self-service join button
private	Channel members only	Creator sends an invitation
direct	Exactly two members	Created with both members enrolled atomically

The channel list page shows all public channels (so workspace members can discover and join them) plus any private and direct channels the user already belongs to. Non-member users who navigate directly to a private or direct channel URL receive a 403 response.

10.3 Search Scoping

The keyword search applies both membership checks in a single query, ensuring results are bounded by the intersection of workspace membership and channel membership:

```
results = Message.objects.filter(
    body__icontains=query,
    channel_id__in=member_channel_ids,
    channel__workspace_id__in=member_workspace_ids,
)
```

This mirrors query C7 from Part 1 and guarantees that a message is only returned if the user belongs to both the workspace *and* the specific channel where it was posted.

11 AJAX Features

11.1 Motivation

Without AJAX, users must manually refresh the page to see new messages. This is particularly disruptive in a chat context. Two features were identified as requiring a live feel: message display and emoji reactions. Other interactions (channel creation, workspace management) use standard form-based POST/redirect/GET because they are infrequent and a full page load is acceptable.

11.2 Message Loading and Polling

On page load, the channel view's JavaScript calls `/channels/<id>/messages/json/`, which returns a JSON array of all messages with reaction data. After the initial load, a `setInterval` fires every three seconds to poll for new messages:

```
async function poll() {
  if (!lastId) return;
  const msgs = await fetchMessages(lastId);
  msgs.forEach(m => {
    if (!document.querySelector(`.msg-row[data-id="${m.id}"]`)) {
      list.appendChild(renderMessage(m));
      if (m.id > lastId) lastId = m.id;
    }
  });
}
setInterval(poll, 3000);
```

The `after` query parameter limits the server response to messages with `message_id>lastId`, so each poll transfers only new messages rather than the full history.

11.3 Message Posting

When the user presses **Send** (or **Enter**), a `fetch()` POST is made to `/channels/<id>/post/` with the message body encoded as `application/x-www-form-urlencoded`. The server creates the `Message` row and returns the serialised message as JSON. The client appends it to the DOM immediately without waiting for the next poll cycle.

11.4 Emoji Reactions

A fixed set of six emoji is supported: thumbs-up, thumbs-down, heart, laughing face, party popper, and fire. Only used emoji (those with at least one reaction) are shown as pill buttons beneath each message. A **+** button opens a floating picker panel containing all six options.

Clicking an existing reaction pill or selecting from the picker sends a POST to `/messages/<id>/react/`. The server implements toggle semantics using the database's unique constraint:

```
try:
```

```
    MessageReaction.objects.create(
        message=message, user=user, emoji=emoji)
    reacted = True
except IntegrityError:
    MessageReaction.objects.filter(
        message=message, user=user, emoji=emoji).delete()
    reacted = False
```

If the row does not exist, the `INSERT` succeeds and the reaction is added. If it already exists, the unique constraint raises an `IntegrityError`, which the application catches and converts into a `DELETE`, removing the reaction. The server then returns the updated per-emoji counts so the client can re-render the reaction row without a full page reload.

11.5 CSRF Handling for AJAX

Django's CSRF middleware requires a token for all non-`GET` requests. For AJAX calls made from JavaScript, the token is read directly from the `csrftoken` cookie set by the middleware and passed as the `X-CSRFToken` header:

```
function getCsrf() {
  const match = document.cookie.match(/csrftoken=([^;]+)/);
  return match ? match[1] : '';
}
// Used in every fetch POST:
headers: { 'X-CSRFToken': getCsrf(), ... }
```

This is Django's recommended pattern for AJAX CSRF protection and avoids the need to embed the token in every rendered HTML template.

12 Security

12.1 SQL Injection Prevention

All database interactions go through the Django ORM, which issues parameterised queries via the psycopg v3 driver. User-supplied values are passed as query parameters, never interpolated into SQL strings. For example, the channel message query:

```
Message.objects.filter(
    body__icontains=query,
    channel_id__in=member_channel_ids,
    ...
)
```

The ORM translates `body__icontains=query` into a prepared statement with a bound parameter, regardless of what `query` contains. No raw SQL strings with user data appear anywhere in the codebase.

12.2 Cross-Site Scripting (XSS) Prevention

Django's template engine escapes HTML entities by default. Every template variable (e.g. `{{ msg.body }}`) is automatically escaped unless explicitly marked safe with the `| safe` filter. The `| safe` filter is never applied to user-supplied content in any template.

For content rendered client-side via JavaScript (message bodies, nicknames, timestamps), the `renderMessage` function passes all values through an `escHtml` helper before inserting them into `innerHTML`:

```
function escHtml(s) {
  return String(s)
    .replace(/&/g, '&amp;').replace(/</g, '&lt;')
    .replace(/>/g, '&gt;').replace(/"/g, '&quot;');
}
```

This prevents a message body containing `<script>` tags from executing in other users' browsers.

12.3 CSRF Protection

Django's `CsrfViewMiddleware` is enabled globally. All HTML forms include a `{% csrf_token %}` tag, and all AJAX POST requests include the `X-CSRFToken` header as described in Section 11.

12.4 Password Storage

Passwords are hashed with Django's default algorithm, PBKDF2-HMAC-SHA256, using a random per-user salt. Plaintext passwords are never stored or logged. The `check_password` function handles constant-time comparison to mitigate timing attacks.

12.5 Atomicity for Multi-Step Writes

Any operation that requires multiple related database writes is wrapped in `@transaction.atomic()` to guarantee all-or-nothing behaviour:

- **Workspace creation:** INSERT into `workspace` and INSERT into `workspace_membership` (admin row) are atomic. If the membership insert fails, the workspace row is rolled back.
- **Channel creation:** INSERT into `channel` and INSERT into `channel_membership` (creator row, plus the second member for direct channels) are atomic.
- **Invitation acceptance:** GET-OR-CREATE `WorkspaceMembership` and marking the invitation `accepted = true` are atomic, preventing a user from accepting the same invitation twice and gaining duplicate memberships.

13 Design Decisions

Polling vs. WebSockets. Three-second polling was chosen over WebSockets for simplicity. WebSockets require a persistent connection, a compatible server (e.g. Daphne), and a more complex client-side reconnection strategy. Polling is stateless and works with the standard Django development server. The three-second interval is short enough to feel responsive while remaining inexpensive at the expected class-demo scale. A production deployment would replace polling with Django Channels and WebSockets.

Fixed emoji set. Rather than allowing arbitrary Unicode emoji input, a fixed set of six (thumbs-up, thumbs-down, heart, laughing face, party popper, fire) is enforced on both the client and server. The server rejects any emoji not in the allow-list, preventing malformed or excessively large strings from reaching the database. The client restricts the picker UI to the same set. This simplifies rendering and avoids font support issues across operating systems.

Composite primary keys in Django 6. Django 4.x automatically adds a surrogate `id` column to every model unless `primary_key=True` is set on another field. For M:N tables with natural composite keys (`workspace_membership`, `channel_membership`), this would diverge from the Part 1 schema. Django 6.0 introduced `CompositePrimaryKey`, which maps the ORM directly onto the existing composite PK columns without adding surrogate columns:

```
class WorkspaceMembership(models.Model):
    pk = CompositePrimaryKey('workspace_id', 'user_id')
    workspace = models.ForeignKey(...)
    user      = models.ForeignKey(...)
    ...
    class Meta:
        managed = False
        db_table = 'workspace_membership'
```

Unmanaged models (`managed = False`). All models carry `managed = False`, which tells Django never to `CREATE`, `ALTER`, or `DROP` the underlying tables. The schema is owned entirely by `finalProjSchema/schema.sql`; Django is only a query layer. This preserves a clean separation between the relational design artifact (Part 1) and the application code (Part 2).

Direct message deduplication. When a user creates a direct channel with another user, the application generates a canonical name by sorting the two usernames lexicographically: `dm-{smaller}-{larger}`. If a channel with that name already exists in the workspace, the user is redirected to it rather than creating a duplicate. This avoids the need for a separate `UNIQUE` constraint on the pair of participants.

14 User Guide

14.1 Registration and Login

Navigate to `/register/` to create an account. You must supply an email address (unique), a username (unique, used for `@mentions` and DM lookups), a display nickname, and a password

of at least six characters. After registering, the application logs you in automatically.

To log in to an existing account, visit `/login/` and enter either your username or email address along with your password. To log out, click **Log out** in the navigation bar.

Profile settings (change nickname or password) are available at `/profile/`.

14.2 Workspaces

Creating a workspace. From the dashboard (`/`), click **New workspace**. You become the admin of the new workspace immediately.

Inviting members. From the workspace page, admins can click **Invite member**. Enter the invitee's email address. If they already have an account, they receive a pending invitation at `/invitations/`. If they do not yet have an account, the invitation is stored against their email and automatically linked when they register.

Managing members. Workspace admins see **Remove** and **Promote** buttons next to each member on the workspace page. An admin cannot remove themselves.

14.3 Channels

Browsing channels. The channel list (`/workspaces/<id>/channels/`) shows all public channels in the workspace and any private or direct channels the user already belongs to. Public channels the user has not joined show a **Join channel** button.

Creating a channel. Click **New channel** and choose a type:

- **Public**: any workspace member can discover and join.
- **Private**: invite-only; only the creator can invite others.
- **Direct**: opens a one-on-one channel with another workspace member. Start typing a username into the autocomplete field to find the person.

Private channel invitations. The creator of a private channel can visit `/channels/<id>/invite/` to send an invitation by username. Pending invitations appear in the channel list under "Pending invitations." The invitee can accept or decline.

14.4 Messaging

Open a channel to view its message history. Messages load automatically via AJAX and new messages appear within three seconds without a page refresh. To send a message, type in the input box at the bottom and press **Send** or **Enter**.

14.5 Emoji Reactions

Beneath each message a **+** button opens a floating emoji picker containing the six supported emoji. Click one to add a reaction. The emoji pill appears under the message with a count badge. Clicking

an existing reaction pill toggles your reaction on or off. All reaction updates are reflected live without a page reload.

14.6 Search

The search bar in the navigation header searches message bodies across all channels and workspaces the current user can access. Results are sorted chronologically and link to the workspace and channel where each message was posted. Messages in workspaces or channels the user does not belong to are never returned.